



CNCF Mentorship 2025

Project Proposal

By Moemen Mahmoud

Project Name

Flatcar Container Linux: Cloud-Init to Butane YAML config transpiler

Information

moamenhamous@gmail.com | [GitHub](#) | [LinkedIn](#)

Experience in Open Source

Pull Request & Analysis Investigation	Description	Status
#7226	fix: parse edited device YAML correctly	under review
#7196	bug: kadm ctl edit device fails with invalid character when parsing edited YAML	under review(open)
#50584	enhancement: Consistent Scalar Types in RunEndEncodedBuilder	Merged
#4820	Bug Fix: Memory alignment in WASM	Analysis / nvestigation

Other Contributions

LibreOffice Open Source Contributor(*Gerrit / CI Contributions*)

(Google Summer of Code 2026)

- 1) Contributed to LibreOffice CI pipeline (Gerrit review system)

Why are you interested in this program?

I am highly interested in this program because it offers the opportunity to **learn and grow through working** in a professional **internship** environment on real-world engineering problems. I am particularly attracted to its **innovative and technically challenging projects** that solve practical problems with **real impact**, which aligns strongly with my interests in systems programming, problem solving, and continuous technical growth.

A little background about myself

I am a **Computer Engineering** student with a strong foundation in Data Structures, Algorithms, Databases, Object-Oriented Programming, and Software Design Patterns. I have hands-on experience in systems and backend development using **C++ and C#**.

My experience includes building a **thread-safe Buffer Pool Manager** in C++, where I worked on memory management, caching, concurrency control, and database internals, strengthening my understanding of low-level system design and performance-critical engineering.

I also developed a **Bank Transaction Control System** using OOP principles, improving my system design and software architecture skills. Additionally, I built a **Smart Map Location Explorer** using .NET MVC and SQL Server, gaining experience in backend development, database design, and API integration.

I am particularly interested in this project because it strongly aligns with my background in **systems programming, backend development, and low-level system design**. Although I have not worked extensively with technologies such as **KubeEdge, RuntimeClass, Kata Containers, and Confidential Containers**, I am fully prepared to learn and work with them. I am a **fast learner** and comfortable working with unfamiliar technologies, so learning the project-specific technologies is not a concern for me. I see this project as an opportunity to expand my systems and cloud-native knowledge while contributing to a meaningful real-world engineering problem.

Project Understanding

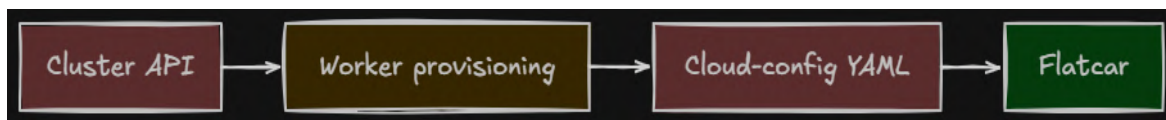
This project will develop a Go based transpiler that converts a defined subset of cloud config YAML into Butane YAML. It will focus on the configuration features needed to provision worker nodes in Cluster API on Flatcar.

The main goal is to make cloud-config configurations usable with Flatcar's Butane-based configuration workflow, providing a simple compatibility layer between the two formats.

Current State , Before the Project

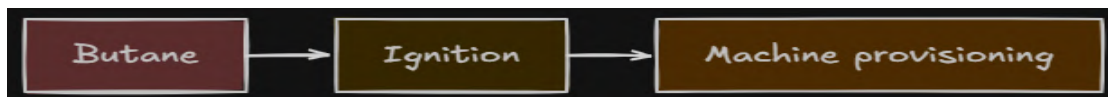
Input: the provisioning configuration is provided as a **cloud-config YAML document** for the worker node.

Current Output / Workflow:



Problem / Gap: the problem is that **cloud-config** and Flatcar's native provisioning workflow use different configuration formats.

Flatcar uses:



while the provisioning configuration may be expressed as: **Cloud-config YAML**

therefore the required conversion layer is missing:



Target State , After the Project:

Input: **Cloud-config YAML**

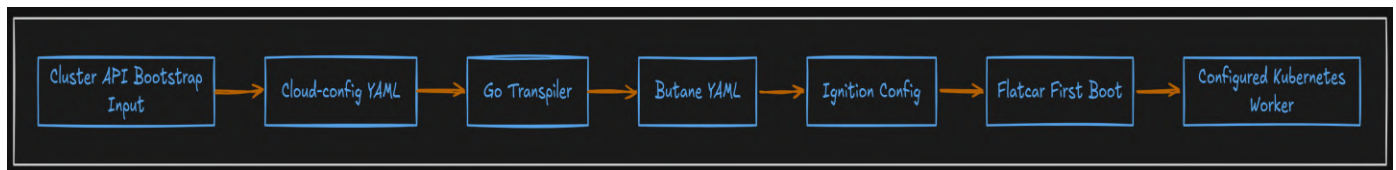
The transpiler accepts a **documented subset of cloud-config YAML** containing the configuration required for Cluster API worker provisioning, including supported features such as users, groups, certificates, files, and systemd units.

Transformation:



The **Go-based transpiler** converts the supported **cloud-config configuration** into its corresponding **Butane YAML representation**.

Output / Final Workflow:

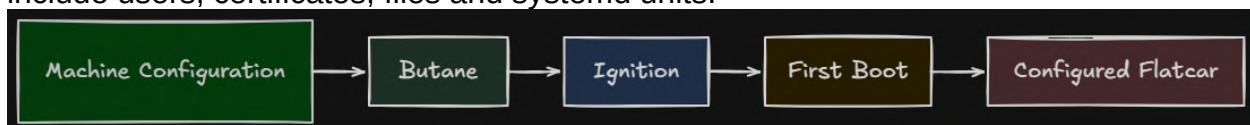


the project introduces a **Go-based transpiler** that converts the **Cloud-config YAML** used for worker provisioning into **Butane YAML**, allowing the configuration to continue through Flatcar's existing Butane to Ignition provisioning workflow.

Understand the Project Environment

Flatcar: is Container Linux is a **Linux based** operating system designed to run containerized workloads. it focuses on providing a minimal and reliable environment that can be updated easily.

unlike operating systems that directly use **cloud config**, flatcar uses **Butane** and **Ignition** for machine provisioning. It follows an immutable operating system model, where the base system is managed separately from the machine configuration. during the initial boot, Ignition applies the configuration to the machine. this configuration can include users, certificates, files and systemd units.



Ignition: is the provisioning system used by Flatcar to configure the machine during the **initial boot process**. it runs early in the boot process, before the system reaches its normal operational state, and applies the configuration that defines the desired state of the machine.

In this project, **ignition** does not consume **cloud-config** directly. instead the configuration follows this path: **cloud config YAML to Go transpiler to Butane YAML to Ignition configuration**. ignition then applies the configuration during **the first boot** of Flatcar, creating and configuring resources such as files, users, groups, certificates and systemd units.



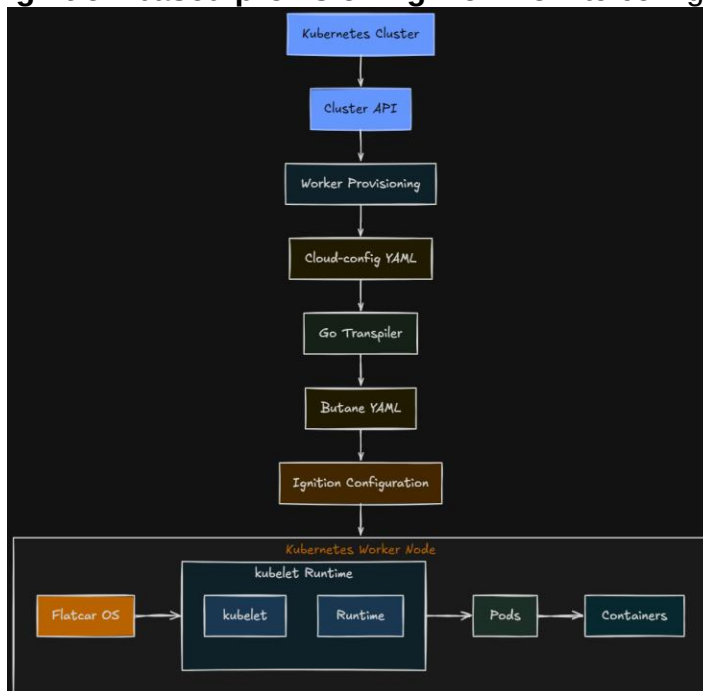
Butane defines the desired machine configuration, while **Ignition** applies this configuration during the **initial boot of Flatcar**.

Butane : is a **configuration language and transpiler** used to define **machine configuration** for systems such as **Flatcar**. It provides a **human-readable YAML** format for describing the desired state of a machine, including users, files, certificates, and systemd units.

Butane then transpiles this configuration into an **Ignition** configuration that can be applied during the machine's **initial boot**.

Cluster API Context and Worker Provisioning:

the project is intended to support **Cluster API worker provisioning on Flatcar**. at a high level, **Cluster API provisions** the worker node, while Flatcar uses its **Butane and Ignition-based provisioning workflow** to configure the machine.



Butane & Ignition Deep Dive

Butane to Ignition Transformation: Butane provides a **human-readable YAML** format for defining the **desired machine configuration**. It is then **transpiled** into an **Ignition configuration**, which **Ignition** can use during the **Flatcar provisioning process**.

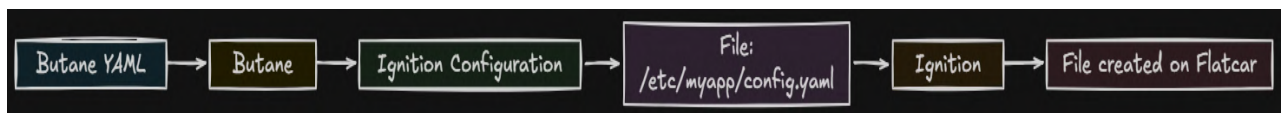
example: File Configuration

A file can be defined in Butane as part of the desired machine configuration

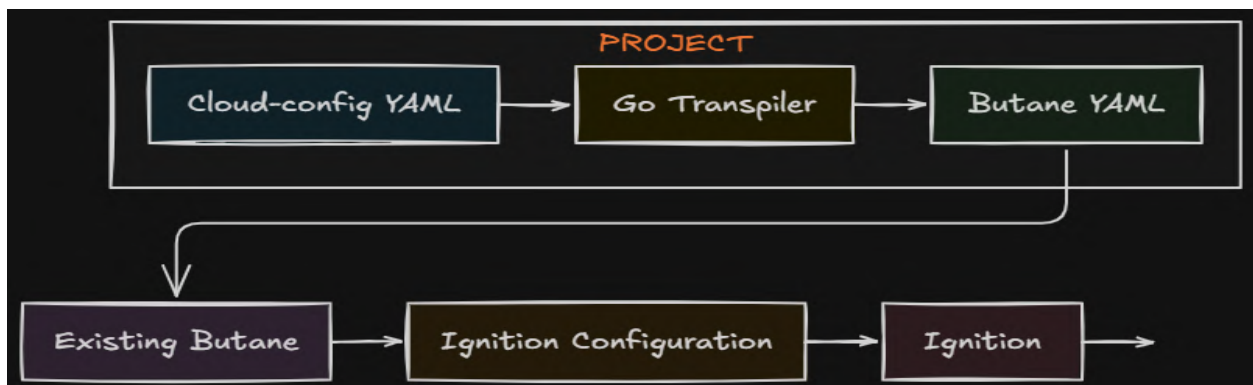
YAML

```
variant: flatcar
version: 1.1.0
storage:
  files:
    - path: /etc/myapp/config.yaml
      mode: 0644
      contents:
        inline: |
          environment: production
          port: 8080
```

this **Butane configuration** is then **transpiled** into the **corresponding Ignition configuration**, which represents the file and its properties in a format that Ignition can apply during provisioning.



Project Boundary and Transformation Flow:



Therefore, the project introduces a new **cloud-config** to **Butane transformation**, while the existing **Butane** workflow handles the subsequent **Butane to Ignition** transformation.

this sets a clear **project boundary**: the Go **transpiler** only needs to generate valid **Butane YAML**, it does not need to reimplement the existing **Butane to Ignition** conversion.

Supported Cloud-config Subset and Semantics

the project will focus on a deliberately limited subset of **cloud-config** required for **Cluster API** worker provisioning on Flatcar. Based on the project scope, the initial supported features to be analyzed are:

- **Users , Groups , Certificates**
- **Files** (`write_files`) , **Systemd units**

rather than targeting full **cloud-config** compatibility, the analysis will identify the relevant fields and configuration semantics within each supported feature and determine the behavior that must be preserved by the **transpiler** for the worker provisioning use case.

for example, for users, this includes relevant attributes such as identity, group membership, shell, and SSH access, for files, file path, content, ownership, permissions and encoding and for systemd units, unit definition, enablement and startup behavior. this analysis will establish the required configuration semantics before mapping them to **Butane**, while keeping the **transpiler** focused on the minimum functionality required by the project and making unsupported functionality explicit.

Cloud-config to Butane Compatibility & Semantic Analysis

I will first look at how the cloud-config features needed for Cluster API worker provisioning can be represented in Butane.

the initial scope covers:

- **Users** → `passwd.users`
- **Groups** → `passwd.groups`
- **Files** (`write_files`) → `storage.files`
- **Certificates** → Butane TLS/certificate configuration
- **Systemd-related configuration** → Butane `systemd.units`

for each feature, I will compare the cloud-config and Butane representations and check:

- Required and optional fields
- Default behavior
- Validation rules and Necessary transformations
- Differences in semantics and unsupported fields and limitations

I have already reviewed the relevant cloud-init and Butane schemas, implementations, and translation tests. This gave me an initial understanding of which fields can be mapped directly and which ones need additional handling because cloud-init and Butane do not always provide equivalent behavior.

these findings will guide the implementation of the transpiler, especially the conversion rules, special cases, supported fields, and test cases. Any remaining differences or edge cases will be checked during implementation and integration testing.

Cluster API Worker Provisioning Analysis

Objective:

Analyze how **Cluster API** generates and **delivers** worker bootstrap configuration and identify the integration point for Flatcar configuration conversion.

Bootstrap Data Generation:

The investigation starts from: `cluster-api/bootstrap/kubeadm`

The worker bootstrap generation path was identified in:
`bootstrap/kubeadm/reconcilers/kubeadmconfig/kubeadmconfig_controller.go`

The worker flow contains:

```
bootstrapJoinData, _, err = ignition.NewNode()
bootstrapJoinData, err = cloudinit.NewNode()
```

the worker bootstrap generation path branches depending on the configured format. for **Ignition-based** Flatcar provisioning, **Cluster API** calls `ignition.NewNode()`. for the default **cloud-init path**, it calls `cloudinit.NewNode()`.

for the **Flatcar/Ignition path**, the relevant flow is:



Bootstrap Data Storage

After generating the worker bootstrap payload, the controller stores it through:
`r.storeBootstrapData(ctx, scope, bootstrapJoinData)`

The generated data is associated with the Machine bootstrap configuration through:
`spec.bootstrap.dataSecretName`

therefore, the relevant delivery path identified from the implementation is:



This establishes that the bootstrap configuration generated by the Bootstrap Provider is persisted as bootstrap data before it is consumed by the infrastructure provisioning workflow.

Flatcar Configuration Generation

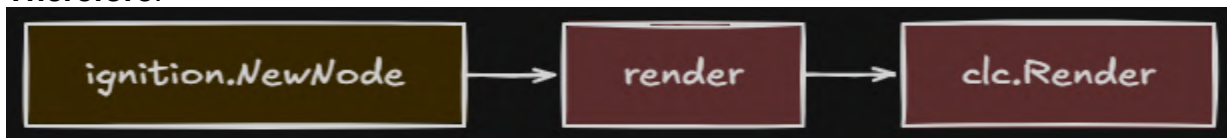
For the Flatcar/Ignition path, the configuration generation was traced to:
`bootstrap/kubeadm/pkg/ignition/`

The main entry point is: `bootstrap/kubeadm/pkg/ignition/ignition.go`

The worker function is: `func NewNode(input *NodeInput)`

It eventually calls: `return render(&input.BaseUserData, input.Ignition, input.JoinConfiguration)`
and `render()` calls: `return clc.Render(input, clcConfig, kubeadmConfig)`

Therefore:



is the relevant Flatcar configuration generation path.

Current Flatcar Conversion Path

the existing implementation imports the **Flatcar Container Linux Config Transpiler**:
github.com/flatcar/container-linux-config-transpiler/config

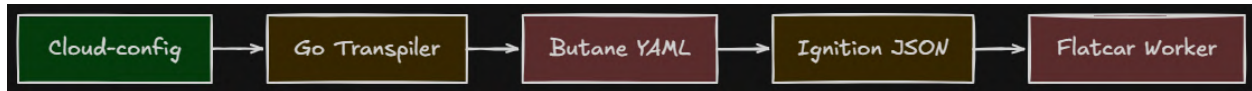
and the Flatcar Ignition configuration package: github.com/flatcar/ignition/config/v2_3

The current conversion path is therefore:



this is the **existing upstream path identified from the repository**, rather than an assumed workflow.

Proposed Project Integration : the project is intended to introduce a different configuration conversion path:



The repository analysis identifies the existing CLC conversion layer as the relevant integration area:

`bootstrap/kubeadm/pkg/ignition/clc/clc.go`

specifically the path through: `clc.Render()`

Therefore, the purpose of the proposed **transpiler** is **not to change the Cluster API bootstrap lifecycle itself**. the relevant change is the configuration conversion stage used for Flatcar.

Cloud-config / Bootstrap Configuration Source

the investigation identified the shared bootstrap configuration structure in:

`bootstrap/kubeadm/pkg/cloudinit/cloudinit.go`

the file defines the **BaseUserData** structure, which contains the common worker bootstrap configuration inputs used during bootstrap generation.

the structure includes configuration categories such as:

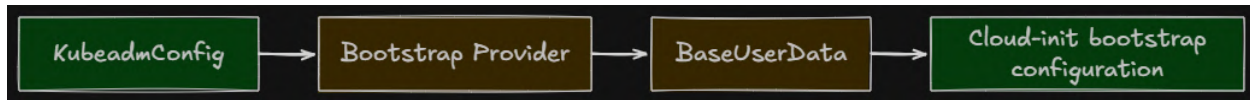
- Users , Files , Boot commands
- Mounts , Disk setup , NTP configuration

The file also defines the cloud-config header:

```
cloudConfigHeader = `## template: jinja
#cloud-config
`
```

this shows that the worker bootstrap configuration is generated during the Cluster API bootstrap workflow rather than being provided as an external **cloud-config file**.

The relevant flow is:



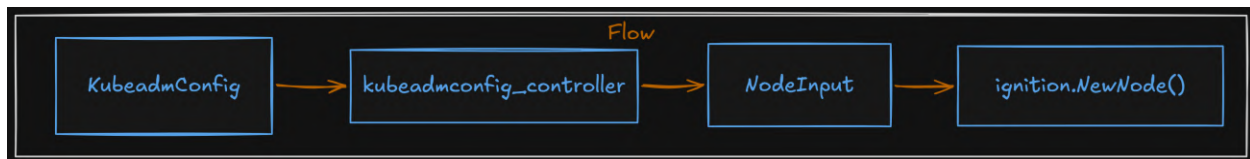
Key finding: Cluster API assembles the worker bootstrap configuration through the shared BaseUserData abstraction, which contains the cloud-config semantics required for worker provisioning.

User-data Ownership and Passing Flow

the next step is to identify which component owns the worker **user-data** and passes it to the **Flatcar/Ignition generation path**.

the investigation identified the worker bootstrap generation logic in:
`bootstrap/kubeadm/reconcilers/kubeadmconfig/kubeadmconfig_controller.go`

the controller prepares the worker bootstrap input and passes it through NodeInput to the Ignition generation path:



for **Flatcar/Ignition-based provisioning**, `ignition.NewNode()` is the entry point responsible for generating the Ignition-based bootstrap payload.

Key finding:

the **Cluster API Bootstrap Provider** owns the worker bootstrap generation workflow. The `kubeadmconfig_controller` prepares the required input and passes it to `ignition.NewNode()` for **Flatcar/Ignition provisioning**.

User-data Consumption in Flatcar/Ignition Path

The worker configuration consumption path was traced to:
`bootstrap/kubeadm/pkg/ignition/ignition.go`

the **Flatcar/Ignition** generation entry point is: `func NewNode(input *NodeInput)`

The configuration is then passed to the Flatcar conversion layer:



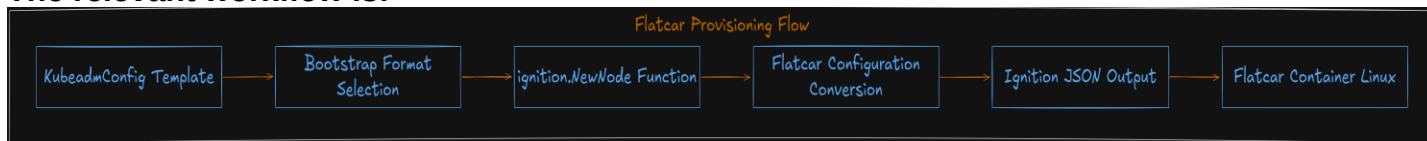
Key finding: the **worker configuration** is consumed by the **Flatcar Ignition generation** path through `ignition.NewNode()`. the existing Flatcar conversion layer is `clc.Render()`, which is the relevant point before Ignition configuration generation.

Flatcar Identification and Transpiler Integration Point

the investigation verifies whether Cluster API explicitly identifies Flatcar as the target operating system during worker provisioning.

the analysis shows that Cluster API does not perform OS detection during the bootstrap workflow. Instead, the Flatcar specific path is reached through the Ignition-based bootstrap flow and the existing Flatcar configuration conversion layer.

The relevant workflow is:



Based on the existing workflow, the new cloud-config to Butane transpiler will integrate at the configuration conversion stage.

Existing Butane Codebase & Integration Analysis

The existing Butane codebase was analyzed to identify the appropriate integration points for the Cloud-init → Butane transpiler.

Butane Configuration Representation

The Flatcar v1.1 configuration builds on the existing `base/v0_5.Config` representation, which provides the structures required for:

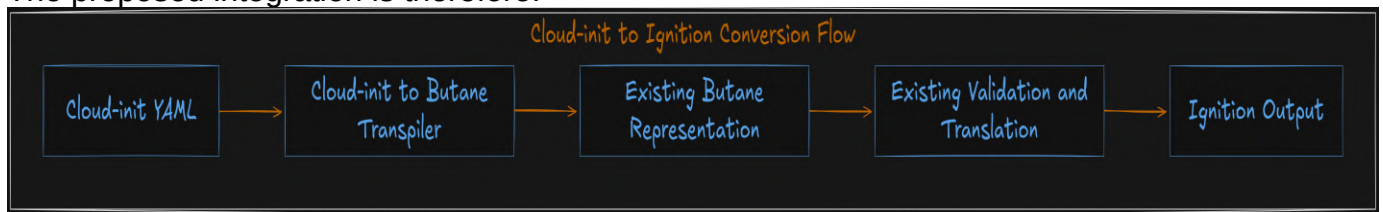
- Users ,Groups , Files
- Systemd units , Certificates

The transpiler will map supported Cloud-init constructs into these existing Butane structures rather than introducing a parallel configuration model.

Validation and Translation Integration

The existing Butane infrastructure already provides validation and translation from the Butane configuration model to Ignition.

The proposed integration is therefore:



This allows the implementation to reuse the existing validation and translation mechanisms instead of introducing a separate validation or translation pipeline.

Butane YAML Output

the existing codebase provides **YAML serialization** for the Butane configuration. The output path uses the existing translation result and YAML encoder to produce Butane YAML.

Therefore, the target output of the transpiler is:



Cloud-config Analysis (Input Side)

The Cluster API Kubeadm Bootstrap Provider generates worker bootstrap data as Cloud-config/user-data. The shared BaseUserData structure is used to assemble the configuration before it is rendered into the bootstrap payload.

For this project, the transpiler will support only the following Cloud-config features within the defined scope:

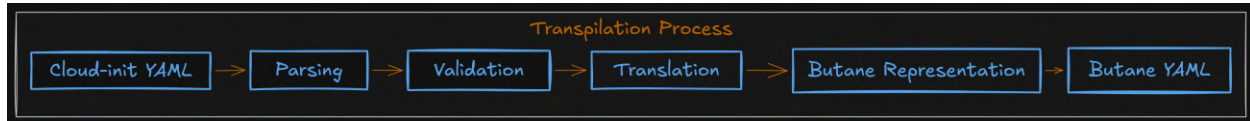
- Users , groups , write_files
- System , certificates

The input boundary is therefore: Cloud-config to Cloud-config to Butane Transpiler

The detailed field mapping for these supported features will be defined during the implementation phase.

Technical Implementation Strategy

Transpiler Pipeline : the transpiler will follow a simple pipeline that converts the supported Cloud-init configuration into valid Butane YAML while reusing the existing Butane representation and downstream translation infrastructure.



- **Parsing:** Parse the Cloud-init YAML into the Go representation used by the transpiler.
- **Validation:** Validate the input structure and ensure that only supported Cloud-init constructs and values are accepted.
- **Translation:** Convert the supported Cloud-init features into their corresponding Butane fields, applying required field-name and semantic transformations.
- **Butane Representation:** Construct the existing Butane configuration structures identified during the codebase analysis.
- **Butane YAML:** Serialize the resulting Butane configuration using the existing YAML output mechanism.

The transpiler will therefore focus on the **Cloud-init parsing, validation, and translation layers**, while reusing the existing Butane configuration representation and output infrastructure.

Cloud-init - Butane Translation Design

The translation layer will implement the mappings identified during the Cloud-init and Butane analysis.

Cloud-init feature	Butane representation	Translation approach
Users	passwd.users	Direct mapping + required field transformations
Groups	passwd.groups	Direct mapping + group membership handling
Files	storage.files	Mapping with content/ownership/permission transformations
Systemd units	systemd.units	Explicit handling of semantic differences
Certificates	security.tls	Mapping of supported certificate configuration

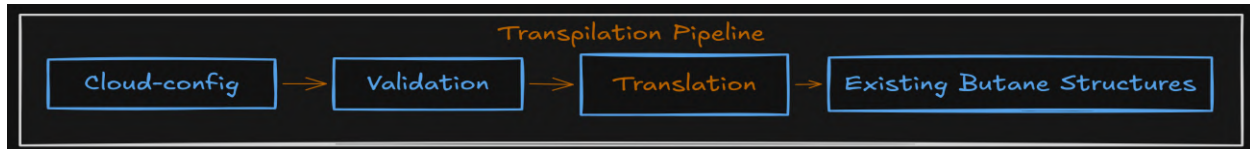
The translation layer will distinguish between:

- **Direct mappings**, where Cloud-init and Butane provide equivalent fields.
- **Field transformations**, where the same concept is represented using different field names or structures.
- **Semantic differences**, where additional conversion logic is required.
- **Unsupported constructs**, where no valid Butane representation exists.

The resulting design will preserve the semantics of the supported Cloud-init subset while preventing unsupported configuration from being silently discarded.

Internal Architecture Decision

based on the Cloud-init and Butane analysis, the transpiler will use **direct translation into the existing Butane configuration structures**.



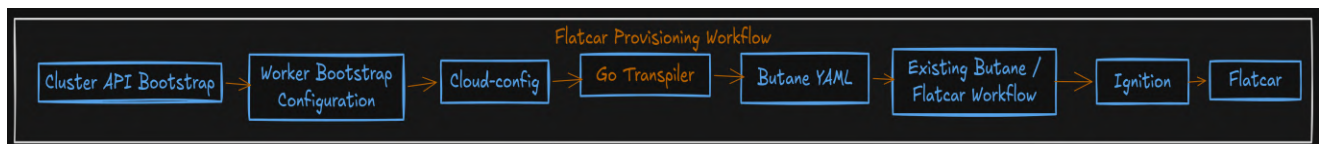
This approach is preferred because:

- **Simplicity:** avoids introducing an unnecessary intermediate representation.
- **Maintainability:** reuses the existing Butane models and keeps the new code focused on Cloud-init to Butane conversion.
- **Testability:** translation functions can be tested directly against the resulting Butane structures.
- **Semantic correctness:** the mapping targets are the existing Butane structures

An intermediate representation will therefore **not be introduced unless implementation reveals a concrete requirement for one**.

Cluster API / Flatcar Integration

The transpiler will be integrated at the **configuration conversion stage** of the existing Cluster API Flatcar provisioning path.



The existing **Cluster API bootstrap lifecycle remains unchanged**. The new component is responsible only for converting the supported Cloud-init configuration into Butane YAML before it continues through the existing Flatcar provisioning workflow.

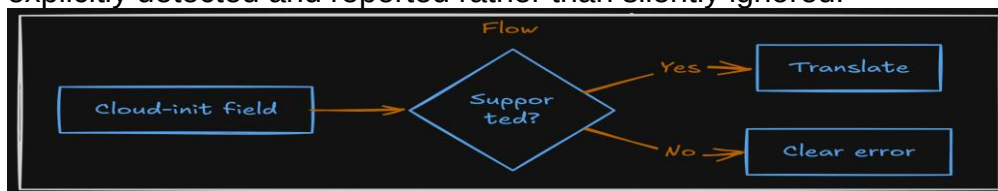
The integration point identified during the repository analysis is the **existing Flatcar configuration conversion layer**, specifically the path currently going through `clc.Render()`.

Validation & Error Handling : define how the **transpiler** validates input and handles unsupported or invalid Cloud-init configurations.

Input Validation:the transpiler will validate the Cloud-init input before translation to ensure that:

- The YAML structure is valid and Required fields are present.
- Field values satisfy the expected constraints.
- Only the supported Cloud-init subset is processed.

Unsupported Features: Cloud-init fields outside the supported subset will be explicitly detected and reported rather than silently ignored.

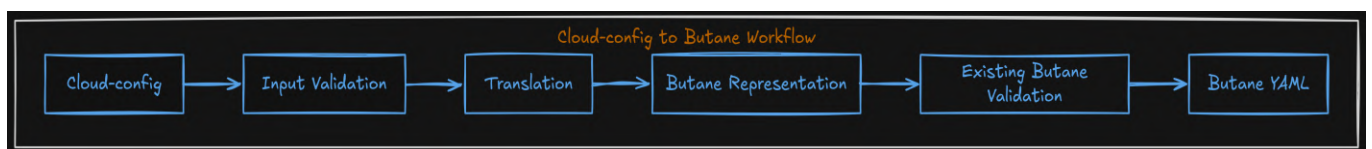


this ensures that unsupported configuration does not result in incomplete or misleading Butane output.

Semantic Validation :the transpiler will check relevant semantic constraints where a Cloud-init configuration cannot be represented correctly by the target Butane model.

if the required semantics cannot be preserved, the transpiler will report an error instead of generating an incorrect configuration.

Butane Validation :after translation, the generated Butane representation will be validated using the existing Butane validation mechanisms before producing the final YAML output.



the transpiler will therefore reuse the existing Butane validation infrastructure rather than introducing a separate validation system.

Deliverable :a validation and error-handling strategy that rejects invalid input, explicitly reports unsupported features, detects relevant semantic incompatibilities, and reuses existing Butane validation for the generated configuration.

Reproducible Provisioning Validation

Objective: Reproduce the relevant Cluster API [to](#) Flatcar worker provisioning path and validate the integration point identified during the repository analysis.

Activities:

- Set up a reproducible Cluster API + Flatcar provisioning environment.
- Verify the existing worker bootstrap and Flatcar/Ignition flow.
- Integrate the Go transpiler at the identified configuration conversion point.
- Verify that generated Butane configuration can continue through the existing Butane [to](#) Ignition workflow.
- Validate successful Flatcar worker provisioning with the converted configuration.

Deliverable: a reproducible provisioning environment and validation results demonstrating that the Go transpiler integrates correctly with the existing Cluster API/Flatcar provisioning workflow.

Implementation Plan

Phase & Dates	Focus Area	Key Tasks & Deliverables
Phase 1: Transpiler Core & Input Validation Sep 7 – Sep 26	Cloud-init Parsing & Validation	-Implement the Go-based Cloud-init input model and parser for the supported subset. -Define and validate the supported configuration and its constraints. -Establish the translation structure for Users, Groups, Files, Systemd units, and Certificates.
Phase 2: Cloud-init to Butane Translation & Validation Sep 27 – Oct 20	Feature Mapping & Butane Generation	-Implement the supported Cloud-init to Butane mappings using the existing Butane structures. -Handle required field transformations, semantic differences, and unsupported configurations. -Reuse existing Butane validation and YAML generation mechanisms.
Phase 3: Cluster API / Flatcar Integration & Final Validation Oct 21 – Nov 7	Integration & End-to-End Provisioning	-Integrate the transpiler at the identified Cluster API/Flatcar configuration conversion stage. -Connect the generated Butane configuration with the existing Butane to Ignition workflow. Validate end to end Cluster API worker provisioning on Flatcar and address integration issues.